

# DocScribe

## Multilingual OCR Pipeline

Hello

こんにちは

नमसे

Hola



Bonjour

你好

मरति



Team ID

25



Company Name

BeyondData



Name

**Khiloni Patel**

202402626020161

**Riya Shah**

202402626020160



# THE PROBLEM

Documents need to have their textual content extracted reliably.

Extracted text must be consumable by downstream systems such as:

- 🔍 Search indexing
- 🤖 Data-entry automation
- 🛡️ Compliance checks
- 📊 Analytics

The absence of a robust, language-aware OCR pipeline creates challenges in processing:

- Multilingual documents
- Structurally complex documents
- These documents may fail to process or produce low-quality, unreliable text output



# LITERATURE REVIEW

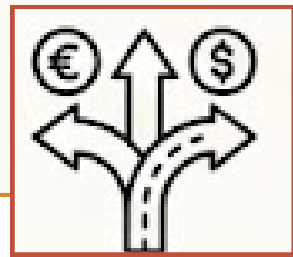
What existing system do?

| Tool                          | Core Tech               | License                          | Key Limitation                                |
|-------------------------------|-------------------------|----------------------------------|-----------------------------------------------|
| Tesseract / PaddleOCR         | LSTM / CRNN             | Apache 2.0 — free                | No adaptive routing; weak on complex layout   |
| Surya / DeepSeek-OCR          | Vision-Language Models  | GPL-3.0 / GPU-bound              | GPU-heavy; can hallucinate on complex visuals |
| Docling (IBM)                 | RT-DETR + TableFormer   | MIT — free                       | No per-page confidence routing                |
| Marker (Datalab)              | Surya VLM + pdftext     | GPL-3.0, paid above \$2M revenue | Library only; no hallucination guard          |
| MinerU (OpenDataLab)          | 2-stage VLM + UniMERNet | AGPL-3.0                         | Heavy (16–32GB RAM); CJK/scientific-focused   |
| Cloud APIs (Google/AWS/Azure) | Proprietary             | Paid — \$1.50–\$70/1,000 pages   | Closed-source; data leaves your infra         |



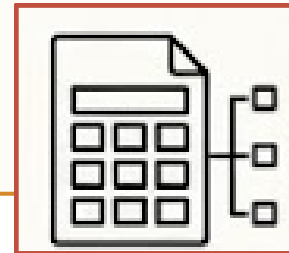
6 categories reviewed — none combine open licensing + adaptive routing + deployable packaging.

# THE RESEARCH GAP → WHERE DOCSCRIBE FITS



Dynamic Cost  
Optimization

No adaptive routing  
between cheap and  
expensive engines



Metadata Preservation

Markdown-first  
design loses  
confidence, bbox,  
structure



End-to-End System

Every open-source  
option is a library,  
not a deployable  
system



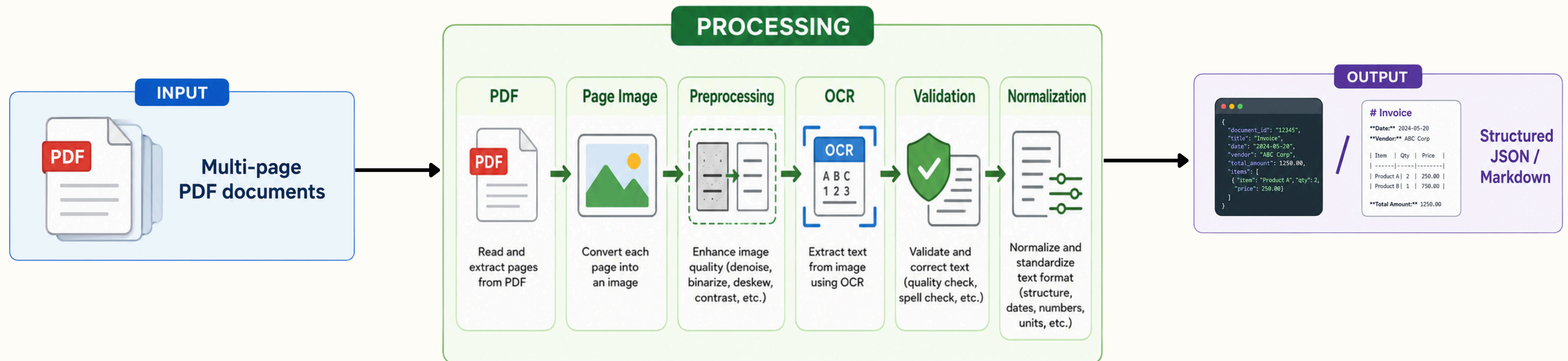
Visual Grounding &  
Guardrails

No explicit  
safeguard against  
hallucinated chart  
content

# THE SOLUTION – DOCSCRIBE

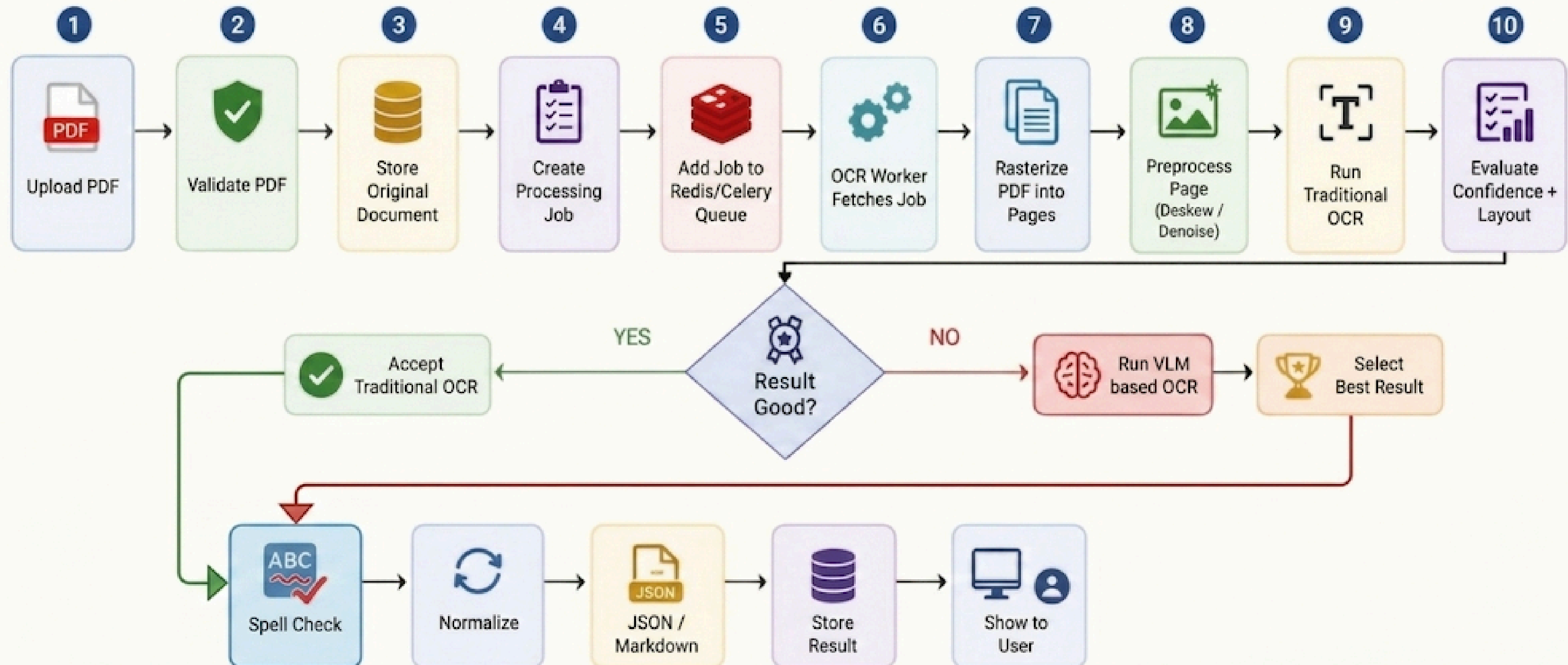
DocScribe is an end-to-end open-source OCR pipeline designed to:

An end-to-end, open-source OCR pipeline that accepts multi-page PDF documents as input and produces clean, structured, and language-correct extracted text as output, suitable for direct use by downstream systems without manual correction.



PDF Upload → Page/Image → Preprocess → Traditional OCR → Confidence Check → VLM based OCR → Normalize → JSON/Markdown

# WORKFLOW DIAGRAM – END-TO-END DOCUMENT WORKFLOW



# WHY PRE-TRAINED MODELS, NOT FROM-SCRATCH?

**Project scope:** The objective is an end-to-end OCR pipeline, not a new OCR model.

**Training cost:** Building OCR from scratch requires large labeled datasets and significant GPU resources.

**Proven foundation:** Traditional based OCR + VLM based OCR already provide strong OCR capabilities for text detection and recognition.

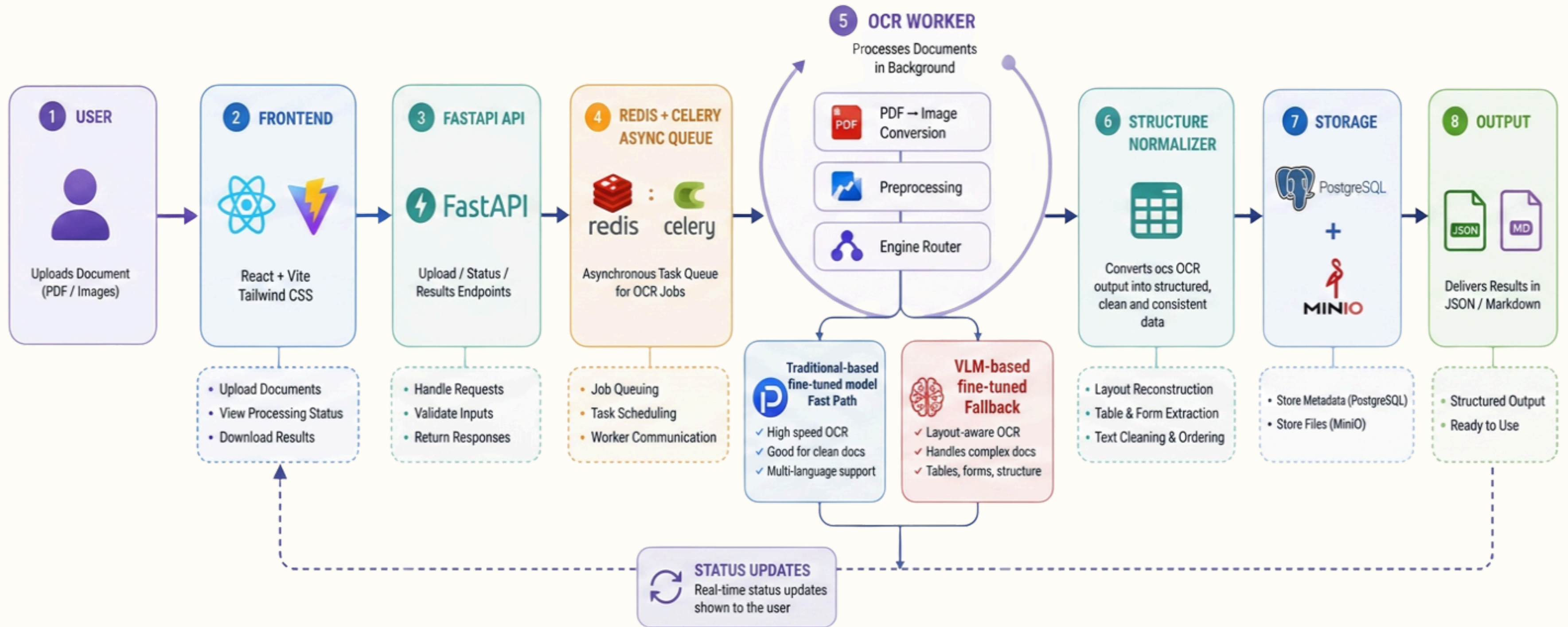
**Our contribution:** We build the intelligence and processing layer around these models:

- Image preprocessing
- Dual-engine OCR routing
- Confidence-based decision making
- Language / mixed-script handling
- Structured JSON output
- Table, image and chart handling

**Industry approach:** Real-world systems typically combine specialized models with application-level intelligence rather than rebuilding every component.

**Future extension:** If domain-specific accuracy requires it, OCR from scratch can be added later.

# SYSTEM ARCHITECTURE



# HYBRID OCR STRATEGY

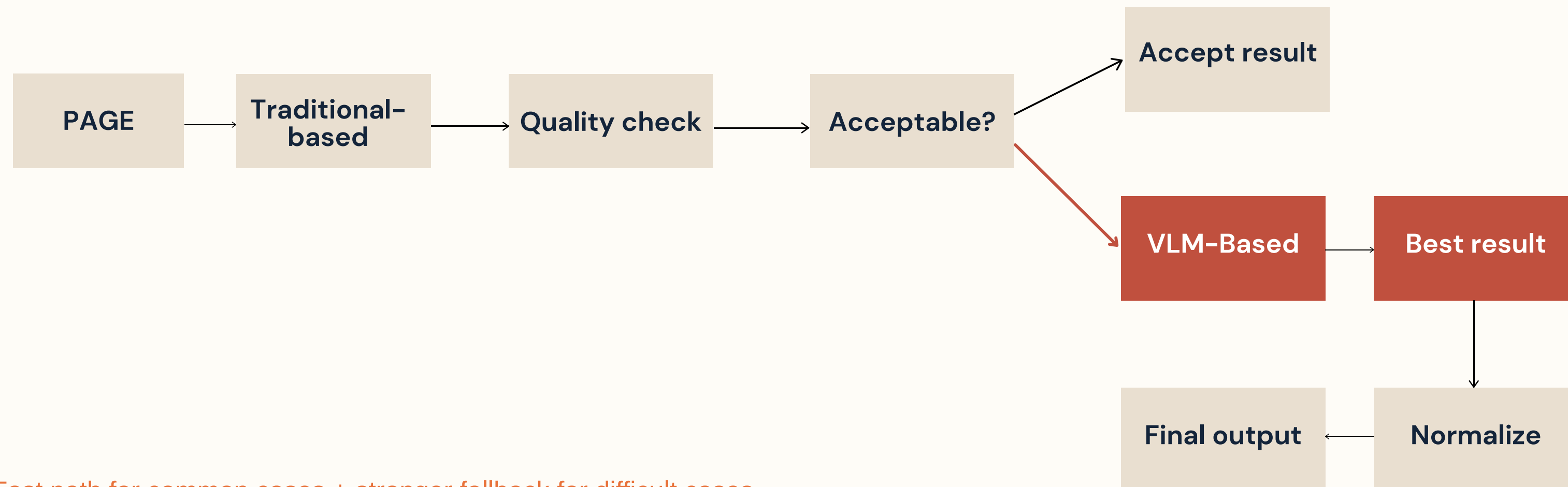
## “Why Hybrid OCR?”

### Traditional-based **Primary Engine**

- Fast
- Suitable for normal pages
- CPU-friendly
- Handles the majority of pages

### VLM-based **Fallback Engine**

- Used for difficult pages
- Useful for complex layouts
- Used when primary OCR results are not reliable
- More computationally expensive



Fast path for common cases + stronger fallback for difficult cases.

# COMPARISON- TRADITIONAL OCR

| Model            | Key Features                                                                                                                            | Base Model / Architecture                                                                                                                                                                            | Best For                                                                             | Limits                                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>PaddleOCR</b> | Multilingual OCR; text detection + recognition; document layout; tables, formulas, handwriting through PP-Structure; CPU/GPU deployment | <b>PP-OCR family</b> — specialized detection + recognition architecture. Classic PP-OCR uses DB text detection + CRNN recognition; newer PP-OCRv6 uses PPLCNetV4, RepLKFPN and LightSVTR components. | Production OCR, invoices, scanned PDFs, multilingual documents, structured documents | More configuration/tuning; best accuracy generally benefits from GPU.                                                |
| <b>Tesseract</b> | Mature open-source OCR; 100+ languages; CPU-first; LSTM recognition; huge ecosystem                                                     | <b>LSTM-based OCR engine</b> since Tesseract 4; not a VLM/LLM                                                                                                                                        | Simple printed text, bulk document digitization, CPU-only systems                    | Weak with handwriting, complex layouts, tables and highly structured documents; GPU support is limited/experimental. |

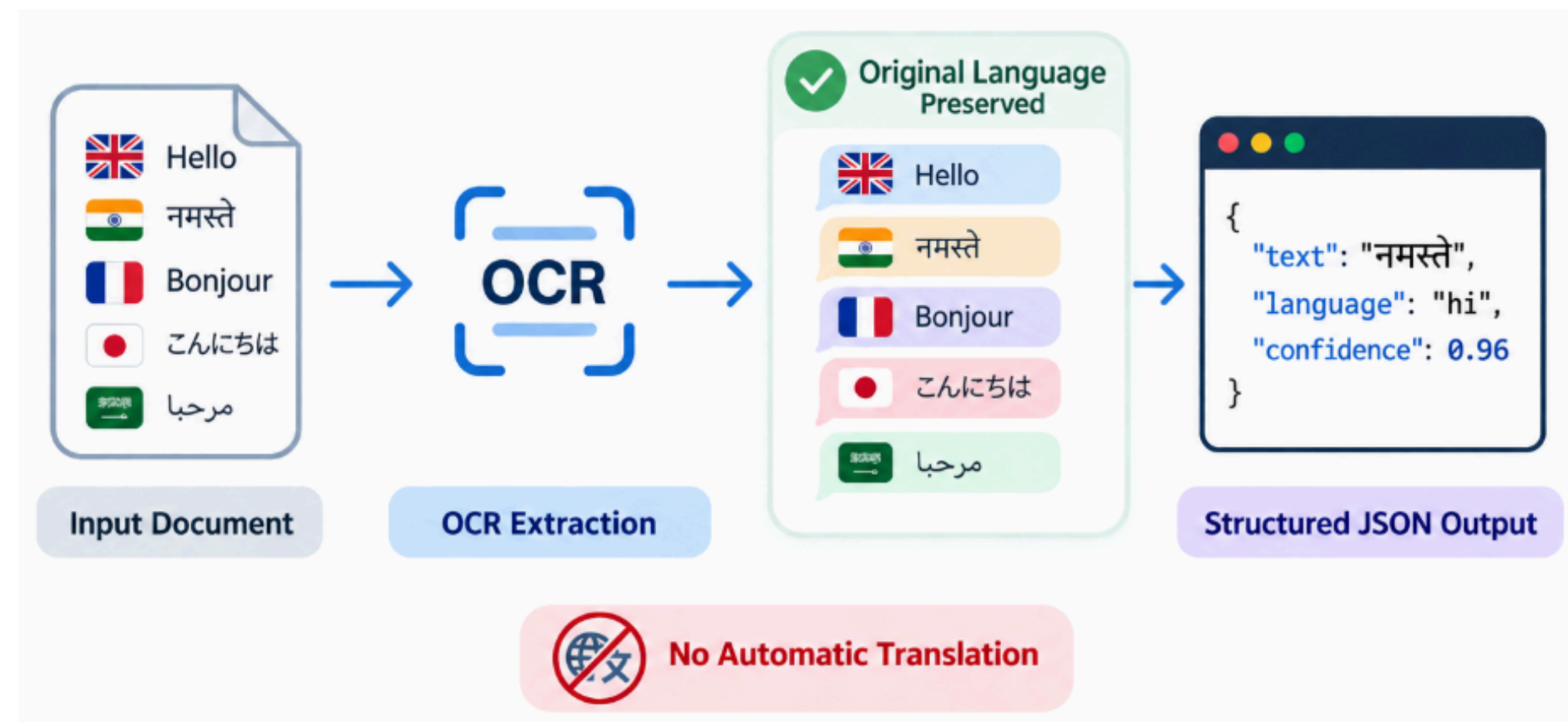
# COMPARISON- VLM BASED OCR

| Model        | Key Features                                                                                                                                     | Base Model / Architecture                                                                                                                                         | Best For                                                                                    | Limits                                                                                                                                             |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| DeepSeek-OCR | End-to-end multimodal OCR; visual token compression; handles text, layout, charts and formulas; transformer/VLM approach                         | DeepSeek-3B MoE decoder + DeepEncoder (~380M). DeepEncoder combines SAM-base 80M + CLIP-Large 300M, followed by a 3B MoE decoder with ~570M activated parameters. | Complex documents, high-throughput GPU OCR, charts/formulas, document understanding         | GPU practically required; can occasionally hallucinate, particularly with overlapping/ambiguous document elements.                                 |
| Surya OCR    | OCR + layout analysis + reading order + table recognition; 90+ languages; document-focused; current Surya 2 unifies OCR/layout/table recognition | Surya 2: ~650M parameter VLM, Qwen3.5-style architecture; older Surya versions used separate recognition/detection models.                                        | PDF/document OCR, complex layouts, tables, multilingual documents, reading-order extraction | More computationally demanding than lightweight OCR; current Surya 2 needs an inference backend such as vLLM/llama.cpp for OCR/layout/table tasks. |

# MULTILINGUAL & STRUCTURED CONTENT HANDLING

## 1. Multilingual Storage

- Preserve the original language of extracted content
- No automatic translation
- Store language information with the OCR output
- Translation, if required, can be handled as a separate future step



# MULTILINGUAL & STRUCTURED CONTENT HANDLING

## 2. Tables → JSON as Source of Truth

- JSON preserves rowspan & colspan
- Stores per-cell confidence
- Preserves complex table structures
- Markdown is generated from JSON for viewing/export
- Markdown is treated as a lossy display format, not the authoritative data

```
Json {  
  "block_id": "p3_b2",  
  "type": "table",  
  "bbox": [x0, y0, x1, y1],  
  "confidence": 0.91,  
  "table": {  
    "rows": 4,  
    "cols": 3,  
    "cells": [  
      { "row": 0,"col": 0, "row_span": 1, "col_span": 1, "text": "Item",  
        "is_header": true, "confidence": 0.95},  
      {"row": 0, "col": 1, "row_span": 1, "col_span": 2, "text":  
        "Details", "is_header": true, "confidence": 0.93}  
    ]  
  }  
}
```

# MULTILINGUAL & STRUCTURED CONTENT HANDLING

## 3. Images & Charts crop + store, don't hallucinate content

### Images

- Detect figure regions
- Crop and store the image separately
- Store a reference to the image in JSON
- Attach nearby captions when available

### Charts

- Will not going to extract chart values using OCR
- Mark the region as chart
- Set needs\_review: true
- This avoids generating potentially incorrect data

```
json {  
  "block_id": "p5_b1",  
  "type": "figure",  
  "subtype": "image | chart",  
  "bbox": [x0, y0, x1, y1],  
  "confidence": 0.88,  
  "image_url": "minio://results/doc123/p5_b1.png",  
  "caption": "Figure 2: Regional sales breakdown",  
  "alt_text": null,  
  "needs_review": true  
}
```

# OUR USP



**Confidence-routed** hybrid engine



**JSON-first**, not Markdown-first



**No chart hallucination** — flagged, never guessed



**Script-preserving** multilingual output



**Deployable system** — queue, storage, DB, UI included

# COMPARISION

| Capability                               | Traditional | VLM-only | Structured<br>(Docling/Marker) | Cloud APIs | DocScribe |
|------------------------------------------|-------------|----------|--------------------------------|------------|-----------|
| Adaptive per-page routing                | ✗           | ✗        | ⚠                              | ✗          | ✓         |
| JSON as source of truth                  | ✗           | ⚠        | ⚠                              | ✓          | ✓         |
| Chart hallucination guard                | ✗           | ✗        | ✗                              | ✗          | ✓         |
| Fully open + self-hosted                 | ✓           | ⚠        | ⚠                              | ✗          | ✓         |
| Deployable system (not library/API-only) | ✗           | ✗        | ✗                              | ✓ (paid)   | ✓         |

# TECH STACK

| Layer          | Technology                  | Why It Is Used                                                                     |
|----------------|-----------------------------|------------------------------------------------------------------------------------|
| Frontend       | React + Vite + Tailwind CSS | Build an interactive interface for PDF upload, job status, and OCR results         |
| Backend        | FastAPI                     | Provide the API and connect the frontend with the Python-based processing pipeline |
| Job Queue      | Celery + Redis              | Run OCR tasks asynchronously without blocking the API                              |
| Primary OCR    | PaddleOCR/Tesseract         | Provide a fast OCR path for normal document pages                                  |
| Fallback OCR   | Surya OCR / VLM-based       | Handle difficult pages where the primary OCR result is unreliable                  |
| PDF Processing | PyMuPDF / pdf2image         | Convert PDF pages into images for OCR processing                                   |
| Preprocessing  | OpenCV                      | Improve scanned-page quality through deskewing, denoising, and binarization        |
| Language ID    | fastText / PaddleOCR        | Identify the document/page language for language-aware processing                  |
| Database       | PostgreSQL                  | Store structured data such as documents, jobs, pages, and OCR metadata             |
| Object Storage | MinIO / S3                  | Store uploaded PDFs and generated processing outputs                               |
| Deployment     | Docker Compose              | Run and manage the project's multiple services in a consistent environment         |

# CONCLUSION

**The core system** — API, database, job queue, storage, and frontend — is already built and working end-to-end

**What remains is focused:** wiring in the two real OCR engines, preprocessing, and block-merging logic

Once complete, **DocScribe becomes a reusable OCR backbone** for search indexing, data-entry automation, and compliance workflows

**Expected impact:** reduces the manual-correction bottleneck for multilingual document processing — with the potential to reduce dependency on paid cloud OCR APIs.

**"DocScribe turns multilingual OCR from a manual bottleneck into a scalable, structured pipeline."**

**THANK YOU !**